FINAL YEAR PROJECT DEFENSE HANDBOOK

SolarPredict

Solar Energy Forecasting and Microgrid Optimization for Douala and Maroua, Cameroon

A complete technical explanation of the data, machine-learning pipeline, application architecture, source code, testing strategy, limitations, demonstration, and oral defense.

Project type
Machine Learning + Full-Stack Web Application

Prediction model
XGBoost Model II

Backend
FastAPI + SQLite

Frontend
React + Vite + Recharts

Weather source
Open-Meteo at runtime

Training source
NASA POWER hourly data

Prepared on
15 June 2026

Project location
SolarPredict repository

Replace this line with your full name, matricule, department, institution, supervisor, and academic year before submission.

How to Use This Guide

This document is both a technical report and a defense study guide. Read Sections 1-5 first to understand the project as a story. Then study Sections 6-14 with the source code open. Finally, rehearse Sections 17-20 before the oral defense.

> **The one-sentence explanation:** SolarPredict uses current weather and time information to predict hourly solar irradiance for Douala and Maroua, stores the results, visualizes them in a web dashboard, and uses them to simulate battery, solar, and grid operation.

Table of Contents

1. Project Overview

SolarPredict is a location-specific solar forecasting and decision-support platform for two climatically different Cameroonian cities: coastal Douala and far-northern Maroua. It combines historical data science with a live web application.

2 cities
Douala and Maroua have separate trained models and climate-season logic.

21 features
Calendar, weather, cyclic time, rain/wind transforms, and seasonal indicators.

7 API routes
Health, cities, dashboard, prediction, history, recommendation, and optimization.

What the system does

1. The user chooses a city and forecast period.
2. FastAPI requests hourly temperature, relative humidity, wind speed, and precipitation from Open-Meteo.
3. The backend transforms each weather timestamp into the same 21 features used during training.
4. A city-specific XGBoost model predicts `ALLSKY_SFC_SW_DWN`, the all-sky surface downward solar irradiance.
5. Negative predictions are clamped to zero because negative solar irradiance is physically meaningless.
6. Predictions and weather inputs are stored in SQLite.
7. The frontend displays current conditions, charts, history, recommendations, and a microgrid simulation.

Why the project matters

Solar generation is intermittent. Clouds, rain, humidity, time of day, and season affect how much energy is available. Forecasting helps operators decide when to charge a battery, when to use stored energy, and when grid support is needed. The project demonstrates how machine learning can support energy planning in a local Cameroonian context.

> **Key contribution:** Model II avoids using direct and diffuse solar radiation components as predictors. Those variables make historical accuracy look excellent but are generally unavailable as future inputs. Model II therefore provides a more realistic deployment pipeline based on weather and time variables that can actually be obtained for a forecast.

2. Problem, Objectives, and Scope

Problem statement

Solar photovoltaic output changes throughout the day and across weather conditions. In Cameroon, this variation differs strongly between humid coastal Douala and drier Maroua. A microgrid that does not anticipate these changes may waste excess solar energy, over-discharge batteries, or depend unnecessarily on the public grid.

General objective

To design and implement a web-based system that predicts hourly solar irradiance and converts the forecast into practical microgrid operating recommendations.

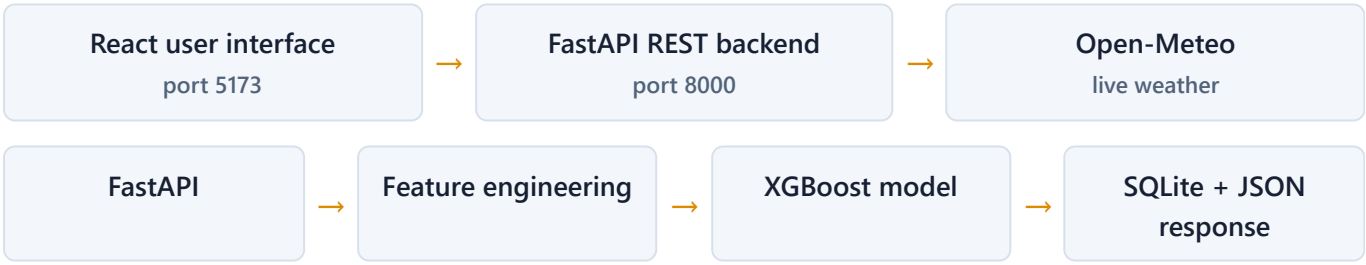
Specific objectives

- Collect and clean historical hourly solar and meteorological data.
- Analyze the relationship between solar irradiance, weather, time, and location.
- Engineer realistic features without unavailable future solar variables.
- Train and compare Random Forest, XGBoost, and LSTM models.
- Deploy the selected model through a FastAPI REST API.
- Build a React interface for forecasts, history, and analytics.
- Simulate battery charging, discharging, grid import, and grid export.

Scope

Included	Not included
Douala and Maroua; hourly forecasts; weather-driven ML; SQLite storage; simple battery/grid simulation; local web deployment.	Physical PV hardware control; electricity tariffs; battery degradation; transmission losses; cloud imagery; probabilistic uncertainty; production authentication; nationwide coverage.

3. System Architecture



Architectural layers

Layer	Technology	Responsibility
Presentation	React, React Router, Recharts, CSS	Forms, navigation, cards, charts, tables, loading and error states.
API/orchestration	FastAPI, Pydantic, Uvicorn	Routes, validation, errors, CORS, calling services, JSON responses, API documentation.
Domain logic	Python modules	Feature engineering, inference, recommendation rules, energy simulation.
External service	Open-Meteo through httpx	Hourly forecast weather for the chosen coordinates.
Persistence	SQLite	Prediction history and dashboard aggregates.
ML artifacts	joblib pickle files	Serialized city-specific XGBoost regressors.

Why FastAPI is used

Browsers cannot directly execute the Python XGBoost model or safely access SQLite. FastAPI acts as the bridge. It receives HTTP requests from React, validates input with Pydantic, calls Open-Meteo and the model, saves results, handles errors, and returns structured JSON. It also automatically provides interactive OpenAPI documentation at `/docs`.

Why separate models per city

Douala and Maroua have different humidity, rainfall, temperature, and seasonal patterns. Separate models let each estimator learn its local relationship between weather/time and irradiance. The trade-off is more model files and maintenance compared with one national model containing location features.

4. Dataset and Preprocessing

Data source

The raw files are NASA POWER hourly data from **1 January 2020 through 1 June 2026** in local solar time. The source combines CERES solar-radiation products and MERRA-2 meteorological products.

City	Coordinates in raw file	Elevation	Raw file lines
Douala	4.0511, 9.7679	103.3 m	56,272 including NASA header text
Maroua	10.5913, 14.3156	417.72 m	56,272 including NASA header text

Original variables

Variable	Meaning	Role
`YEAR, MO, DY, HR`	Calendar year, month, day, hour	Time information
`ALLSKY_SFC_SW_DWN`	All-sky downward shortwave radiation	Prediction target
`ALLSKY_SFC_SW_DNI`	Direct normal irradiance	Exploration/Model I only
`ALLSKY_SFC_SW_DIFF`	Diffuse irradiance	Exploration/Model I only
`T2M`	Temperature at 2 metres, degrees Celsius	Model II input
`RH2M`	Relative humidity at 2 metres, percent	Model II input
`PRECTOTCORR`	Corrected precipitation	Model II input
`WS10M`	Wind speed at 10 metres, m/s	Model II input

`Data_Preprocessing.ipynb`

This notebook performs the first reproducible transformation:

1. Imports pandas, NumPy, Matplotlib, and Seaborn.
2. Reads each NASA file with `skiprows=15` to skip the descriptive header.
3. Inspects shape, columns, data types, missing values, duplicates, and descriptive statistics.
4. Combines `YEAR`, `MO`, `DY`, and `HR` into a proper `datetime`.
5. Checks negative values in the three solar-radiation columns. NASA uses `-999` for missing/unavailable source values.
6. Removes records where solar variables are negative.
7. Saves city-specific cleaned files and a combined file containing a `City` column.

52,584
Cleaned hourly records for Douala.

52,584
Cleaned hourly records for Maroua.

105,168
Combined cleaned observations.

> **Defense detail:** The CSV line count includes one header row, so 52,585 lines mean 52,584 observations. State clearly whether a number includes the header.

5. Exploratory Data Analysis

`Exploratoty_analysis.ipynb`

The EDA notebook answers whether the data are usable and what patterns a model should learn. It performs:

- Shape, type, descriptive-statistics, and null-value inspection.
- Histograms of target irradiance for each city.
- Correlation matrices and target-correlation rankings.
- Per-variable distributions and IQR-based outlier counts.
- Time-series plots of solar radiation.
- Scatterplots between target irradiance and weather/radiation variables.
- A daytime-only analysis that removes the many legitimate zero values occurring at night.
- Comparison of daytime irradiance distributions and hourly means between cities.

Important interpretation

Night-time zeros are not missing data or erroneous outliers. They are physically correct. If all hours are included, the target distribution has a large spike at zero. A daytime-only view is useful for comparing solar-resource strength, but the deployed model still needs night records so that it learns to predict approximately zero outside daylight hours.

What correlations mean

Correlation indicates association, not causation. Direct and diffuse irradiance naturally correlate strongly with total downward irradiance, but using them to predict that total can create leakage because they are components of nearly the same physical quantity. This observation motivated Model II.

6. Feature Engineering

Model I versus Model II

Model I	Model II
Uses direct and diffuse irradiance alongside weather. It achieves extremely high scores but assumes solar variables are available at prediction time.	Excludes direct and diffuse irradiance. It uses only calendar/time and forecast weather, making deployment more realistic.

`Feature_engineering.ipynb`

The initial notebook selects `ALLSKY_SFC_SW_DNI`, `ALLSKY_SFC_SW_DIFF`, temperature, humidity, wind, and precipitation. It creates train/test variables and saves feature datasets. This is the experimental Model I path and remains useful as evidence of iteration.

`feature_model_II.ipynb` and `backend/app/features.py`

The notebook defines the realistic feature set used for training. The backend module reproduces the same transformations for live weather. Training-serving consistency is critical: names, formulas, and column order must match exactly.

Group	Features	Purpose
Calendar	`YEAR, MO, DY, HR`	Long-term and within-calendar position.
Weather	`T2M, RH2M, PRECTOTCORR, WS10M`	Atmospheric conditions available from forecasts.
Daily cycle	`hour_sin, hour_cos`	Represent hour cyclically so 23:00 and 00:00 are close.
Annual cycle	`month_sin, month_cos, day_sin, day_cos, day_of_year`	Represent seasonal movement through the year.
Indicators	`is_daytime, is_rainy`	Explicit daylight and rainfall flags.
Transforms	`rain_log1p, ws10m_log1p`	Compress skewed non-negative variables while preserving zero.
Local season	`season_dry, season_rainy`	City-specific dry/rainy season context.

Cyclic formulas

```
hour_sin = sin(2 * pi * hour / 24)
hour_cos = cos(2 * pi * hour / 24)
month_sin = sin(2 * pi * month / 12)
month_cos = cos(2 * pi * month / 12)
day_sin = sin(2 * pi * day_of_year / 365)
day_cos = cos(2 * pi * day_of_year / 365)
```

A single numeric hour would make 23 and 0 appear far apart. Sine and cosine place them next to each other on a circle. Both are required because sine alone cannot uniquely distinguish all positions.

Season rules

- Douala dry season: December-February; other months are treated as rainy.
- Maroua rainy season: June-September; other months are treated as dry.

`build_model_ii_row()` generates one feature dictionary. `build_feature_dataframe()` converts multiple dictionaries into a pandas DataFrame and enforces the trained column order.

7. Model Training and Evaluation

Training notebooks

- `Model_training_Douala.ipynb` and `Model_training_Maroua.ipynb`: Model I experiments.
- `Model_training_Douala_Model_II.ipynb` and `Model_training_Maroua_Model_II.ipynb`: realistic Model II experiments.
- `Model_comparism.ipynb` and `Model_comparism_Model_II.ipynb`: metric visualizations and model ranking.

Algorithms compared

Algorithm	How it works	Why compare it
Random Forest	Averages many decision trees trained on sampled data/features.	Strong nonlinear baseline, robust, interpretable through feature importance.
XGBoost	Builds trees sequentially, each correcting errors from previous trees using gradient boosting.	Excellent tabular-data performance and efficient inference.
LSTM	A recurrent neural network designed to retain information across sequences.	Tests whether explicit temporal sequences improve forecasting.

Selected XGBoost configuration

```
XGBRegressor(
    objective="reg:squarederror",
    n_estimators=200,
    learning_rate=0.05,
    max_depth=6,
    random_state=42
)
```

The saved Douala and Maroua deployment models both report exactly 21 input features in the expected order.

Evaluation metrics

Metric	Meaning	Better value
MAE	Average absolute prediction error. Easy to interpret in target units and less sensitive to large errors.	Lower
RMSE	Square root of average squared error. Penalizes large misses more strongly.	Lower
R-squared	Proportion of target variance explained relative to a mean baseline.	Closer to 1

Model II results

City	Model	MAE	RMSE	R-squared
Douala	Random Forest	24.806	54.715	0.9476
Douala	**XGBoost**	24.833	**53.323**	**0.9503**
Douala	LSTM	28.581	56.796	0.9434
Maroua	Random Forest	**26.530**	61.109	0.9646
Maroua	**XGBoost**	26.546	**60.195**	**0.9657**
Maroua	LSTM	42.401	74.166	0.9500

XGBoost was selected because it has the best R-squared and RMSE in both cities. Random Forest has a very slightly lower MAE, so the choice is not that XGBoost wins every metric; it provides the preferred overall balance, especially for larger errors and explained variance.

Why Model I scores were higher

Model I XGBoost scores reached approximately 0.9962 R-squared in Douala and 0.9979 in Maroua. Those scores are not directly comparable evidence of better deployment because the predictors include direct and diffuse solar radiation, which are physically close to the target. Model II scores are lower but more credible for real forecasting.

> **Important methodology answer:** The notebooks use a random 80/20 split for tree models. For a time-dependent forecasting problem, a chronological split or rolling-origin evaluation would be stronger because it tests future periods using only past training data. Present the current split honestly and propose time-series validation as a major improvement.

8. Complete Folder Structure

```
SolarPredict/
|-- README.md                          Project summary and run instructions
|-- Data_Preprocessing.ipynb           Raw NASA data cleaning
|-- Exploratoty_analysis.ipynb         Statistical and visual EDA
|-- Feature_engineering.ipynb          Initial/Model I features
|-- feature_model_II.ipynb             Realistic 21-feature pipeline
|
|-- Data/
|   |-- Raw/
|   |   |-- Douala_Dataset.csv         NASA POWER source data
|   |   `-- Maroua_Dataset.csv
|   `-- Processed/
|       |-- Douala_Cleaned.csv
|       |-- Maroua_Cleaned.csv
|       |-- Combined_Cameroon_Solar_Dataset.csv
|       |-- Douala_Features.csv        Model I experiment data
|       |-- Maroua_Features.csv
|       |-- Douala_Features_Model_II.csv
|       `-- Maroua_Features_Model_II.csv
|
|-- training/
```

```
|   |-- Model_training_Douala.ipynb
|   |-- Model_training_Maroua.ipynb
|   |-- Model_training_Douala_Model_II.ipynb
|   |-- Model_training_Maroua_Model_II.ipynb
|   |-- Model_comparism.ipynb
|   |-- Model_comparism_Model_II.ipynb
|   |-- results_*.csv                 Saved evaluation metrics
|   |-- model_ii_comparison_summary.csv
|   `-- saved_models/model_ii/        Training output artifacts
|
|-- Model/
|   |-- douala_xgb_model_ii.pkl       Deployed Douala model
|   |-- maroua_xgb_model_ii.pkl       Deployed Maroua model
|   |-- *_rf_model_ii.pkl             Comparison artifacts
|   |-- *_lstm_model_ii.pkl           Comparison artifacts
|   `-- best_*_xgb.pkl                Model I artifacts
|
|-- backend/
|   |-- requirements.txt              Python dependencies
|   |-- inspect_db.py                 Database inspection utility
|   |-- solar_predict.db              Runtime SQLite database
|   `-- app/
|       |-- __init__.py
|       |-- main.py                   FastAPI routes/orchestration
|       |-- config.py                 Paths, cities, 21 feature names
|       |-- schemas.py                Pydantic request/response contracts
|       |-- weather.py                Open-Meteo async client
|       |-- features.py               Live feature engineering
|       |-- model_loader.py           Model cache and prediction
|       |-- database.py               SQLite schema and queries
|       |-- microgrid.py              Battery/grid simulation
|       `-- services/
|           `-- microgrid_recommendation.py
|
|-- frontend/
|   |-- package.json                  JS dependencies and scripts
|   |-- vite.config.js                Dev server and /api proxy
|   |-- index.html                    Browser entry document
|   `-- src/
|       |-- main.jsx                  React mount + BrowserRouter
|       |-- App.jsx                   Page routes
|       |-- api/client.js             HTTP client
|       |-- styles/index.css          Theme, layout, responsive rules
|       |-- components/
|       |   |-- Layout.jsx
|       |   |-- StatCard.jsx
|       |   `-- RecommendationBadge.jsx
|       `-- pages/
|           |-- Dashboard.jsx
|           |-- Prediction.jsx
|           |-- History.jsx
|           |-- Microgrid.jsx
|           `-- About.jsx
|
`-- docs/
    `-- SolarPredict_Defense_Guide.*  This guide and generated PDF
```

9. Backend Code Explained

`backend/app/config.py`

Centralizes configuration. `BASE_DIR` finds the project root from the file location. `MODEL_DIR` and `DB_PATH` produce portable paths. `CITIES` maps each city to a display name, runtime coordinates, timezone, and model filename. `MODEL_II_FEATURES` is the authoritative 21-column order. `TARGET` names the trained output.

`backend/app/schemas.py`

Defines Pydantic models, which are typed contracts between clients and the API.

- `PredictionRequest`: city and 1-168 hours.
- `WeatherSnapshot`: returned weather and calendar fields.
- `MicrogridAdvice`: status, recommendation, and level.
- `PredictionPoint/Response`: one hour and the complete forecast.
- `DashboardResponse`: current weather, prediction, advice, and aggregate statistics.
- `MicrogridRequest`: positive battery/load/area, efficiency in (0,1], and 1-48 hours.
- `MicrogridHour/Response`: hourly energy balance and totals.

Pydantic rejects invalid inputs before business logic runs and generates API schemas automatically.

`backend/app/weather.py`

`fetch_hourly_weather()` validates the city, builds Open-Meteo query parameters, requests enough forecast days, parses parallel hourly arrays, converts timestamps, and calls `build_model_ii_row()`. It returns records containing a human-facing weather snapshot and a model-facing feature dictionary.

The function is asynchronous because network waiting should not block the server process. `httpx.AsyncClient` has a 30-second timeout and `raise_for_status()` converts failed HTTP statuses into exceptions.

`backend/app/model_loader.py`

`get_model()` validates the city, checks that its model file exists, loads it with `joblib.load()`, and caches it in the module-level `_models` dictionary. Caching avoids loading a model from disk for every request. `predict()` enforces feature order, runs inference, converts NumPy values to Python floats, and clamps results with `max(0.0, value)`.

`backend/app/database.py`

- `init_db()` creates the database directory and ensures the predictions schema.
- `_ensure_predictions_schema()` creates a new table or migrates a legacy schema.
- `get_connection()` is a context manager that guarantees connection closure and returns dictionary-like rows.
- `save_predictions()` inserts many rows efficiently with `executemany()`.
- `get_prediction_history()` supports optional city filtering and a result limit.
- `get_latest_prediction()` retrieves one newest row.
- `get_dashboard_stats()` computes counts, latest records, and average prediction per city.

Queries use placeholders for user-provided values, preventing SQL injection. The selected column names are explicit instead of using `SELECT *`.

`backend/app/main.py`

This is the FastAPI entry point. It creates the application metadata, enables CORS for the two local frontend origins, initializes SQLite and preloads city models at startup, and defines all HTTP routes.

Each prediction route follows the same orchestration pattern: validate city, fetch weather, build a feature DataFrame, run inference, generate recommendations, save results where required, and return a Pydantic response. Expected failures become meaningful `HTTPException` responses.

`backend/inspect_db.py`

A developer utility that prints all SQLite tables, SQL definitions, columns, primary-key/not-null flags, foreign keys, and row counts. It is not part of the web server.

`backend/requirements.txt`

Pins reproducible backend versions: FastAPI/Starlette/Uvicorn for the server, httpx for weather requests, joblib/XGBoost for models, NumPy/pandas for features, Pydantic for validation, and multipart/settings support.

10. API Endpoints

Method	Path	Purpose	Main side effect
GET	`/`	Backend help message and links.	None
GET	`/api/health`	Confirms service status and supported cities.	None
GET	`/api/cities`	Returns city names and coordinates.	None
GET	`/api/dashboard?city=douala`	One live forecast plus dashboard statistics.	Saves one prediction
POST	`/api/predict`	Forecasts 1-168 hours.	Saves every prediction
GET	`/api/history`	Returns up to a requested number of stored rows.	None
GET	`/api/microgrid/recommendation`	Applies threshold rules to one irradiance value.	None
POST	`/api/microgrid/optimize`	Forecasts and simulates a battery/grid schedule.	None in current code

Prediction request example

```
POST /api/predict
Content-Type: application/json

{
  "city": "douala",
  "hours": 24
}
```

Validation and status codes

- `200`: successful request.
- `400`: unsupported city checked by route logic.
- `422`: Pydantic validation error, such as zero hours or negative battery capacity.
- `500`: internal/model/optimization failure.
- `502`: upstream weather or prediction pipeline failure in dashboard/prediction routes.

11. Database Design

Active predictions table

Column	Type	Meaning
`id`	INTEGER PK	Automatically increasing identifier.
`city`	TEXT	Lowercase city key.
`temperature`	REAL	2 m temperature input.
`humidity`	REAL	2 m relative humidity input.
`wind_speed`	REAL	10 m wind speed input.
`precipitation`	REAL	Forecast precipitation input.
`prediction`	REAL	Predicted irradiance.
`timestamp`	TEXT	ISO forecast timestamp.

At documentation time, the runtime database contained 233 prediction rows: 168 for Douala and 65 for Maroua. These counts are live data and will change whenever dashboard or prediction requests are made.

> **Repository observation:** The current database also contains a `microgrid_runs` table with four rows, but the current `database.py` and `microgrid.py` do not write or read it. Treat it as a legacy/previous feature unless persistence is reconnected. Do not claim current optimization runs are saved.

Why SQLite

SQLite is serverless, bundled with Python, simple to deploy, and sufficient for a single-user academic prototype. A production multi-user system would normally use PostgreSQL or another managed database for concurrency, backups, roles, and scaling.

12. Microgrid Recommendation and Optimization

Rule-based recommendation

Predicted irradiance	Status	Action
`> 700`	High Solar Energy	Charge batteries and prioritize solar generation.
`400-700`	Moderate Solar Energy	Use hybrid operation mode.
`< 400`	Low Solar Energy	Use battery reserves or grid support.

`MicrogridRecommendation` is an immutable dataclass. `get_microgrid_recommendation()` returns status, text, and a visual level (`high`, `moderate`, or `low`).

Energy conversion

```
solar_generation_kWh =
    irradiance_W_per_m2 * panel_area_m2 * panel_efficiency / 1000
```

The division by 1000 converts watts to kilowatts, and each forecast step represents one hour, so kW multiplied by one hour gives kWh.

Simulation assumptions

- Initial battery state of charge is 50% of capacity.
- Load is uniform: `daily_load_kwh / hours`.
- Panel efficiency defaults to 18%.
- Charging and discharging are modeled as 100% efficient.
- No maximum charge/discharge power, reserve SOC, degradation, tariff, inverter loss, or export limit is modeled.

Hourly dispatch logic

1. Predict irradiance and convert it to solar energy.
2. Compute `net = solar_generation - hourly_load`.
3. If net is positive, charge the battery up to its capacity; export remaining surplus.
4. If net is negative, discharge available battery energy; import any remaining deficit from the grid.
5. Record battery SOC, import, export, recommendation, and weather timestamp.

Self-sufficiency

```
self_sufficiency_percent =
    min(100, (daily_load - grid_import) / daily_load * 100)
```

This expresses the share of demand not supplied by the external grid. The `min(100)` cap prevents surplus generation from reporting more than complete self-sufficiency.

13. Frontend Code Explained

`frontend/index.html`

Provides the browser document, page title, font links, root `<div>`, and module script that launches React.

`src/main.jsx` and `src/App.jsx`

`main.jsx` mounts React into `#root`, enables `BrowserRouter`, and imports global styles. `App.jsx` defines routes. The shared `Layout` wraps Dashboard, Prediction, History, Microgrid, and About pages.

`src/api/client.js`

Centralizes HTTP communication. `API_BASE` uses `VITE_API_URL` when configured or defaults to `/api`. The generic `request()` function sets JSON headers, catches network errors, parses FastAPI errors, and returns decoded JSON. The exported `api` object provides one method per backend feature.

`vite.config.js`

Runs the frontend on port 5173 and proxies `/api` to `http://127.0.0.1:8000`. Therefore browser code can call relative URLs without CORS complexity during development.

Reusable components

- `Layout.jsx`: responsive sidebar, navigation links, brand, footer, and `Outlet`.
- `StatCard.jsx`: generic metric card accepting label, value, unit, icon, and style variant.
- `RecommendationBadge.jsx`: color-coded recommendation display based on energy level.

Pages

Page	State and behavior
`Dashboard.jsx`	Tracks city, response, loading, and errors. Loads on mount and city change. Shows six cards, update time, total records, average-by-city bar chart, and latest predictions.
`Prediction.jsx`	Collects city and hours, calls `api.predict()`, shows current recommendation, a Recharts line chart, and the first 12 forecast rows.
`History.jsx`	Loads up to 100 database records, reloads when city changes, and shows all weather/prediction columns.
`Microgrid.jsx`	Collects capacity, load, and panel area; sends fixed 18% efficiency and 24 hours; displays summary cards, energy-balance chart, and first 12 schedule rows.
`About.jsx`	Static explanation of purpose, stack, workflow, and recommendation rules.

`src/styles/index.css`

Defines reusable design tokens, dark navy cards, amber accents, layout grids, forms, buttons, tables, status colors, and a responsive breakpoint at 1100 px. Below that width, the sidebar becomes non-sticky and multi-column grids stack into one column.

React concepts to explain

- `useState` stores interactive values and server responses.
- `useEffect` performs side effects such as loading data after render or city changes.
- `useCallback` gives the dashboard loader a stable dependency-aware function identity.
- Conditional rendering shows errors, loading placeholders, or result sections.
- `map()` converts arrays into table rows and chart points.
- Controlled inputs keep form values synchronized with React state.

14. End-to-End Functional Flows

Dashboard flow

1. Dashboard mounts with Douala selected.
2. React calls `GET /api/dashboard?city=douala`.
3. FastAPI requests one hour of Open-Meteo weather.
4. The backend generates 21 features and predicts irradiance.
5. A microgrid status is selected from the thresholds.
6. Existing statistics are queried, then the current record is saved.
7. React renders cards, chart, and latest rows.

> **Small implementation detail:** Dashboard statistics are calculated before the current prediction is saved, so `total_predictions` and latest records in that same response can lag the newly inserted record by one request.

Multi-hour prediction flow

1. User selects city and 1-168 hours.
2. Pydantic validates the request.
3. Open-Meteo returns enough forecast days, sliced to the requested hours.
4. The model predicts all rows in one batch.
5. Each result receives weather details and a recommendation.
6. All rows are inserted with one `executemany()` call.
7. React plots every prediction but displays only the first 12 table rows.

Microgrid flow

The optimization endpoint repeats weather and prediction, then iterates hour-by-hour through the energy dispatch logic. It returns totals plus a schedule. The frontend transforms the schedule into four bar series: solar, load, grid import, and grid export.

15. Installation and Operation

Backend

```
cd backend
python -m pip install -r requirements.txt
python -m uvicorn app.main:app --reload --port 8000
```

Useful backend URLs:

- `http://127.0.0.1:8000/` - help response
- `http://127.0.0.1:8000/api/health` - health test
- `http://127.0.0.1:8000/docs` - interactive Swagger UI
- `http://127.0.0.1:8000/redoc` - alternative API documentation

Frontend

```
cd frontend
npm install
npm run dev
```

Open `http://localhost:5173`. On Windows systems where PowerShell blocks `npm.ps1`, use `npm.cmd run dev` without changing machine-wide execution policy.

Production frontend check

```
cd frontend
npm.cmd run build
```

The build creates `frontend/dist/`. In the verified repository state, 841 modules were transformed successfully. Vite reported a non-fatal warning that the main JavaScript chunk is about 567 kB before gzip; route-level code splitting can reduce it.

Common failures

Symptom	Likely cause	Action
Frontend says backend is unreachable	Uvicorn is not running or wrong URL.	Start backend and test `/api/health`.
Model file not found	`Model/` artifact missing or renamed.	Restore exact city filename from `config.py`.
502 weather/prediction error	No internet, Open-Meteo failure, or incompatible feature/model data.	Inspect FastAPI terminal and test weather API connectivity.
422 validation error	Hours or numeric inputs outside Pydantic constraints.	Read the JSON `detail` field.
Blank/incorrect charts	No result yet or unexpected response schema.	Inspect browser network response and console.

16. Testing Plan and Verified Results

Tests already performed while preparing this guide

Verification	Result
Compile all backend application modules	**PASS** No Python syntax/import compilation errors.
Feature row count and recommendation boundaries	**PASS** 21 features; 701 high, 400 moderate, 399 low.
Load deployment models and inspect schema	**PASS** Both are `XGBRegressor` models with the expected 21 feature names.
Synthetic five-hour Douala inference	**PASS** Five non-negative outputs; noon example about 458.94.
SQLite schema/read check	**PASS** Predictions table readable; city counts and latest rows returned.
React/Vite production build	**PASS** 841 modules transformed and `dist` generated.

No permanent automated test suite currently exists in the repository. The following should be implemented for stronger software-engineering evidence.

Unit tests

Target	Cases	Expected result
Feature engineering	Midnight/noon; leap date; rain zero/positive; both city seasons; feature order.	Correct 21 values, finite transforms, valid flags.
Recommendation	-1, 0, 399.99, 400, 700, 700.01.	Exact low/moderate/high boundary behavior.
Energy conversion	0 and known irradiance/area/efficiency.	Non-negative value matching formula.
Model loader	Known city, uppercase city, unknown city, missing model.	Cache reuse and expected exceptions.
Database	Fresh DB, legacy migration, insert, city filter, limit, aggregates.	Correct schema and rows without modifying production DB.

API integration tests

Use FastAPI `TestClient` or `httpx` and mock `fetch_hourly_weather()` so tests are deterministic and do not require internet.

- `GET /api/health` returns status and both cities.
- Unsupported city returns 400.
- `hours=0` and `hours=169` return 422.
- A mocked 24-hour prediction returns 24 points and saves 24 rows.
- Weather-service exception becomes the documented 502 response.
- Optimization rejects negative capacity/load and efficiency above 1.

Frontend tests

- Render every page using React Testing Library.
- Mock API success, loading, network failure, and validation failure.
- Verify city/hour form values and request payloads.
- Verify result cards, table rows, and recommendation colors.
- Run responsive manual tests at desktop, tablet, and mobile widths.

Model tests

- Recompute MAE, RMSE, and R-squared from a locked test set.
- Use chronological holdout and rolling-window validation.
- Compare errors by city, hour, month, rainy/dry season, day/night, and irradiance range.
- Check prediction distribution, negative raw predictions, and concept drift.
- Test live feature ranges against training distributions.

Manual acceptance test

- [ ] Start backend and confirm models preload without fatal error.
- [ ] Open Swagger and call health and cities.
- [ ] Start frontend and load both city dashboards.

- [ ] Run a 24-hour forecast and verify chart/table/history.
- [ ] Test recommendation values 399, 400, 700, and 701.
- [ ] Run microgrid with small and large battery capacities and compare imports.
- [ ] Stop backend and confirm the frontend presents a useful error.
- [ ] Enter invalid values and verify validation messages.

17. Defense Demonstration Script

Recommended 8-10 minute live sequence

1. **Opening (45 seconds):** State the energy-planning problem, cities, target, and practical output.
2. **Architecture (45 seconds):** Show the folder tree and explain React → FastAPI → weather/model/database.
3. **Data and ML (90 seconds):** Show NASA source dates, preprocessing notebook, 21 Model II features, and comparison CSV. Explain leakage avoidance and why XGBoost was selected.
4. **API proof (45 seconds):** Open `/docs`, call health, and point out request/response validation.
5. **Dashboard (60 seconds):** Switch between Douala and Maroua; explain live weather, prediction, recommendation, averages, and saved history.
6. **Prediction (60 seconds):** Run 24 hours; explain the time curve, night zeros, weather columns, and saved records.
7. **History (30 seconds):** Filter by city to prove persistence.
8. **Microgrid (90 seconds):** Run the default case, explain conversion and dispatch. Increase battery capacity and discuss expected lower grid dependence.
9. **Close (45 seconds):** State results, limitations, and strongest future improvement.

Suggested opening words

> "My project addresses the variability of solar energy in Cameroon. I built SolarPredict, a full-stack decision-support system that forecasts hourly all-sky solar irradiance for Douala and Maroua using weather and time variables. I compared three machine-learning approaches, deployed city-specific XGBoost models through FastAPI, and connected the forecasts to a simplified battery and grid optimization simulator presented through a React dashboard."

What to do if live internet fails

- Do not panic or repeatedly refresh.
- Show `/api/health` to prove the local backend is running.
- Open History to show previously stored predictions.
- Explain that prediction weather is an external dependency and identify the handled 502 path.
- Keep screenshots or a short screen recording of a successful full workflow as backup.

18. Limitations, Risks, and Improvements

A strong defense does not pretend the prototype is perfect. It identifies constraints precisely and proposes technically appropriate improvements.

Current limitation	Why it matters	Improvement
Random train/test split	Can mix earlier and later timestamps and overestimate future generalization.	Chronological holdout, walk-forward validation, and retraining schedule.
NASA training vs Open-Meteo runtime	Different providers and variable definitions create domain shift.	Calibrate/match units and distributions; train with historical data from the deployment provider.
Precipitation semantic mismatch risk	NASA header describes `PRECTOTCORR` in mm/day while Open-Meteo returns hourly precipitation in mm.	Confirm aggregation used by NASA hourly rows; transform runtime precipitation to equivalent scale.
Irradiance unit wording	NASA source labels hourly energy as Wh/m2 while UI/code says W/m2.	Define whether values represent hourly irradiation or average hourly irradiance and use consistent labels/formulas.
Hard-coded daylight 06:00-18:00	Actual sunrise/sunset varies by date and location.	Use solar elevation, sunrise/sunset, latitude, longitude, and day of year.
Simplified microgrid	May overestimate usable battery/solar energy.	Add inverter and battery efficiency, SOC limits, power limits, degradation, tariffs, and uncertainty.
No forecast uncertainty	Operators cannot see confidence or risk.	Prediction intervals, quantile regression, or ensembles.
External weather dependency	Network/API downtime blocks new forecasts.	Caching, retry/backoff, fallback provider, and last-known forecast.
Pickle model serialization warning	Pickles can be version-dependent and unsafe from untrusted sources.	Export XGBoost native JSON/UBJ and record model metadata/version/hash.
No automated test suite	Regressions can reach the application unnoticed.	pytest, mocked API tests, frontend tests, and CI.
SQLite and no authentication	Not suited to secure multi-user production deployment.	PostgreSQL, authentication, authorization, rate limiting, secrets, HTTPS.
Only two cities	Cannot claim national generalization.	Add more climatic zones or train a unified geospatial model.

Other code-level observations

- Move startup handling from deprecated `@app.on_event("startup")` to a FastAPI lifespan context in a future update.
- Move environment-specific origins, URLs, and database paths into settings/environment variables.
- URL-encode city query values in the frontend even though current choices are controlled.
- Add indexes on `predictions(city, timestamp)` as history grows.
- Save microgrid runs only if the UI needs history; otherwise remove the unused legacy table.
- Split React routes/charts into lazy-loaded chunks to address the Vite bundle-size warning.

19. Likely Viva Questions and Model Answers

1. What exactly are you predicting?

> The target is `ALLSKY_SFC_SW_DWN`, all-sky surface downward shortwave solar radiation for each hour. The application presents it as predicted solar irradiance and uses it as the basis for estimating PV energy.

2. Why XGBoost?

> It models nonlinear interactions well on tabular data, handles mixed feature scales without neural-network scaling, provides fast inference, and achieved the best RMSE and R-squared for both cities in Model II. Random Forest had marginally lower MAE, but XGBoost gave the preferred overall error profile.

3. Why is Model II more realistic than Model I?

> Model I uses direct and diffuse solar radiation, which are strongly related components of the target and may not be available for future prediction. Model II uses weather and time information that a forecast service can provide, reducing leakage and enabling actual deployment.

4. Why use sine and cosine for time?

> Time is cyclic. Hour 23 is close to hour 0, but raw numbers make them appear far apart. Sine and cosine encode position on a circle and preserve that continuity.

5. Why not one model for all Cameroon?

> This prototype focuses on two contrasting climates. Separate models learn local relationships directly. A national version should add latitude, longitude, elevation, and climate-zone features and validate whether a unified model generalizes better.

6. What does FastAPI contribute?

> It exposes the Python model and services over HTTP, validates input, handles asynchronous weather requests and errors, coordinates feature engineering and storage, returns JSON, supports CORS, and automatically documents the API.

7. Why SQLite?

> It is lightweight and sufficient for a local academic prototype. It requires no separate database server. For concurrent production use I would migrate to PostgreSQL.

8. How is solar energy converted to kWh?

> For a one-hour interval, I multiply irradiance by panel area and efficiency, then divide by 1000. This assumes the forecast value acts as average power density over that hour. Unit consistency is an important item to formalize because the NASA source labels hourly values as Wh/m2.

9. How does the battery algorithm work?

> The battery begins at 50% SOC. Solar first supplies the hourly load. Surplus charges the battery, then exports to grid. During a deficit, the battery discharges first, and the remaining deficit is imported. SOC is capped between zero and capacity.

10. What is data leakage?

> Leakage occurs when training inputs contain information that would not genuinely be available at prediction time or indirectly reveal the target. It produces overly optimistic evaluation. Excluding direct and diffuse solar components is the main leakage-control decision in Model II.

11. What is the biggest methodological limitation?

> The random train/test split. A chronological or rolling evaluation would better represent forecasting future unseen periods. A second major issue is distribution mismatch between NASA training inputs and Open-Meteo runtime inputs.

12. What does R-squared 0.95 mean?

> On the held-out split, the model explains about 95% of the variance relative to predicting the mean. It does not mean 95% accuracy, and the result depends on the evaluation design.

13. Why can predictions be clamped to zero?

> Solar irradiance cannot physically be negative. Tree regressors can still output small negative values, especially at night, so the post-processing constraint converts them to zero.

14. Why asynchronous weather requests?

> Network calls spend time waiting. Async I/O lets the server handle other work while waiting instead of blocking the worker for the full request duration.

15. How do you know frontend and backend agree on data shape?

> Pydantic response models define backend contracts, while the API client and page mappings consume those fields. Integration tests should lock this contract and catch schema changes.

16. How would you improve accuracy?

> First improve evaluation and data alignment, not just tune hyperparameters. I would use provider-consistent historical forecasts, solar geometry, cloud cover, pressure and visibility, chronological cross-validation, systematic hyperparameter search, and prediction intervals.

17. Is this truly optimization?

> The current implementation is a deterministic rule-based dispatch simulation, not mathematical optimization with an objective solver. It calculates a feasible greedy schedule. A stronger version would formulate cost, emissions, battery wear, and constraints as linear or mixed-integer optimization.

18. What happens when Open-Meteo fails?

> The backend catches the exception and returns an HTTP error, typically 502 in prediction routes, and the frontend displays its message. Production hardening would add retries, caching, and a fallback source.

19. How is reproducibility supported?

> The repository includes raw/processed data, preprocessing and training notebooks, result CSVs, pinned backend dependencies, model artifacts, configuration, and run commands. Stronger reproducibility would add an environment lockfile, random-seed documentation, automated pipeline scripts, and model metadata.

20. What is your original contribution?

> The contribution is the integrated local workflow: realistic city-specific solar forecasting for two Cameroonian climates, a deployable weather-to-feature-to-model pipeline, persistent web visualization, and conversion of predictions into microgrid decision support.

20. Revision Checklist and Glossary

Facts to memorize

- [ ] Target variable name and physical meaning.
- [ ] Four live weather variables and all feature groups.
- [ ] Why Model II excludes DNI and diffuse radiation.
- [ ] XGBoost parameters: 200 estimators, 0.05 learning rate, depth 6, seed 42.
- [ ] Douala XGBoost: MAE 24.83, RMSE 53.32, R-squared 0.9503.
- [ ] Maroua XGBoost: MAE 26.55, RMSE 60.20, R-squared 0.9657.
- [ ] Recommendation thresholds and exact boundary behavior.
- [ ] Energy conversion and self-sufficiency formulas.
- [ ] Every major folder and request flow.
- [ ] Top three limitations and improvements.

Glossary

Term	Meaning
API	A defined HTTP interface through which software components communicate.
CORS	Browser security rules controlling cross-origin requests.
Feature	An input variable used by a machine-learning model.
Target	The output value the model learns to predict.
Inference	Using a trained model to produce predictions.
Regression	Prediction of a continuous numeric value.
Gradient boosting	Sequential models that correct previous residual errors.
Data leakage	Unavailable or target-revealing information entering model inputs/evaluation.
Domain shift	Training and production inputs follow different distributions.
SOC	Battery state of charge.
Grid import/export	Energy bought from or sent to the external grid.
REST	Resource-oriented HTTP API style using methods such as GET and POST.
Serialization	Saving a model object into a file for later loading.

Final defense structure

Always tell the project in this order: **problem → data → leakage-aware features → model comparison → architecture → live functionality → evaluation → limitations → future work.** This order demonstrates that the application is the deployment of a research process, not merely a collection of screens.

> **Final message to the jury:** The project demonstrates an end-to-end engineering pipeline, from historical data and model experimentation to a usable decision-support application. Its strongest quality is the realistic Model II deployment path; its clearest next step is rigorous time-based validation and unit/provider alignment.

Generated from the SolarPredict repository on 15 June 2026. Technical values reflect the inspected code, datasets, model files, result CSVs, and runtime database at that date.